MANUAL DE ENGENHARIA DE SOFTWARE • EDIÇÃO 1.0

AS ✦

Amigo Secreto Inteligente

Relatório técnico completo de construção, arquitetura, replicação, publicação, operação e possibilidades de comercialização

Base documental: código público do projeto e histórico de implementação

Data de referência: 1º de novembro de 2026

Abordagem: prática, técnica e reproduzível • português brasileiro

Repositório: github.com/Leonardocosta3/amigo-secreto

Aplicação: leonardocosta3.github.io/amigo-secreto/

Mapa do documento

Roteiro de referência • capítulos organizados da concepção à venda e operação.

Sumário executivo
1. Evolução da construção do projeto
2. O que o produto faz
3. Arquitetura Full Stack
4. Pré-requisitos e ferramentas
5. Estrutura de diretórios e função dos arquivos
6. Construindo e compreendendo o backend
7. Banco de dados e Prisma ORM
8. Autenticação, autorização e privacidade
9. Motor de sorteio: explicação técnica
10. Sorteio definitivo, transações e consistência
11. Construindo e compreendendo o frontend
12. Execução local passo a passo
13. Versionamento e GitHub
14. Migrando de XAMPP para TiDB Cloud
15. Publicação na internet: a configuração utilizada
16. Pontos críticos encontrados na revisão do código
17. Testes recomendados e critérios de aceite
18. Solução de problemas recorrentes
19. Operação, backups e custos da hospedagem
20. Comercialização: o que é possível vender?
21. Licença MIT, propriedade intelectual e contratos
22. Estratégia comercial sugerida para começar
23. Roadmap de evolução priorizado
24. Procedimento de entrega para outra pessoa
Apêndice A — Referência rápida de comandos
Apêndice B — Glossário essencial
Fontes e referências

Como ler este documento. Trechos identificados como *implementado* foram conferidos na versão pública do repositório ou no histórico da construção. Exemplos simplificados servem para ensinar a ideia sem substituir o código-fonte integral. Trechos identificados como *recomendação* são aprimoramentos ainda não comprovadamente concluídos. Senhas, tokens e credenciais reais não são reproduzidos neste manual.

Sumário executivo

O **Amigo Secreto Inteligente** é uma aplicação web Full Stack, desenvolvida para permitir que organizadores criem eventos de amigo secreto, cadastrem participantes, estabeleçam restrições entre pessoas, realizem um sorteio válido e distribuam resultados individuais. A implementação usa **React 19, TypeScript e Vite 8** no frontend; **Node.js, Express 5, Prisma ORM 7 e MySQL/TiDB** no backend e na persistência; autenticação por **JWT**, senhas protegidas com **bcrypt/bcryptjs**, controles de autorização por proprietário do evento e um algoritmo de sorteio baseado em **busca com retrocesso (backtracking)**.

O percurso efetivamente seguido partiu de um ambiente local com **VS Code + Node.js + XAMPP/MariaDB**, passou por modelagem no Prisma, criação de rotas REST e interface React, correção de conexões e migrações, e terminou numa publicação de demonstração em três serviços: **GitHub Pages (frontend)**, **Render (API)** e **TiDB Cloud Starter (banco de dados)**. O login do organizador e a conexão da API com o banco foram testados com sucesso no sistema publicado.

Este relatório descreve **como reproduzir a arquitetura existente**, **como compreender os arquivos e comandos essenciais**, **como revisar pontos críticos antes de convidar usuários externos** e **como estruturar serviços pagos** com uma operação sustentável.

Elemento	Ambiente / solução efetiva	Papel
Frontend	React 19 + TypeScript + Vite 8	Telas e interação no navegador
Backend	Node.js + Express 5	Regras e API REST
ORM	Prisma 7.10.1 + adaptador MariaDB	Acesso tipado ao banco
Desenvolvimento	MySQL/MariaDB local (XAMPP)	Persistência durante construção
Produção de demonstração	TiDB Cloud Starter compatível com MySQL	Persistência remota
Identidade	JWT + bcrypt/bcryptjs	Login de organizadores e participantes
Publicação da API	Render Web Service	Executa servidor Node.js
Publicação da interface	GitHub Pages + GitHub Actions	Entrega arquivos estáticos
Versionamento	Git + GitHub	Histórico e colaboração

O que está confirmado e o que exige novo teste

Confirmado no percurso: backend no Render respondeu a `/healthz` e `/api/testar-banco`; GitHub Actions publicou o frontend; o organizador conseguiu entrar no site público. **Confirmado no código:** rotas de eventos, participantes, restrições, simulação e sorteio definitivo; autenticação de participantes; modelo de resultados e algoritmo com testes unitários. **Não considerado validado apenas por esses testes:** entrega de links individuais no GitHub Pages, criação de senha do participante, recuperação completa após suspensão do Render, carga simultânea, backup restaurável e toda a superfície de segurança. O manual inclui verificações específicas para esses pontos.

1. Evolução da construção do projeto

O projeto foi criado por incrementos, estratégia apropriada a uma aplicação em que a regra do sorteio é tão importante quanto o visual. A sequência abaixo representa a trajetória técnica conhecida, e não um registro cronológico exato de cada commit.

Etapa	Decisão ou entrega	Por que importou
1. Escopo	Eventos, participantes, restrições e sorteio válido	Transformou uma ideia em requisitos verificáveis
2. API local	Node.js, Express e Prisma	Separou persistência e regras de negócio da interface
3. Banco local	XAMPP com MySQL/MariaDB	Permitiu prototipar sem custo de nuvem
4. Frontend	React + Vite + TypeScript	Interface reativa consumindo a API
5. Integridade	Restrições, backtracking, testes e persistência transacional	Evitou sorteios impossíveis ou incoerentes
6. Identidade	Organizador por evento; acesso do participante	Criou fluxos e permissões distintos
7. Publicação	GitHub Pages, Render e TiDB Cloud	Tornou o projeto acessível de qualquer computador

Os desafios enfrentados também fazem parte do aprendizado: permissões do banco no XAMPP; credenciais e pool de conexões; importação de dados com codificação correta; migrações remotas; publicação de pastas de uma mesma base Git; permissões do executável Prisma no Linux do Render; e instalação das dependências de desenvolvimento para compilar TypeScript.

2. O que o produto faz

2.1 Fluxo do organizador

1. O organizador se autentica por e-mail e senha.
2. Abre o painel **“Meus sorteios”**, visualiza somente eventos associados à própria conta e pode cadastrar um novo evento.
3. Entra em um evento e cadastra participantes, com nome e contatos opcionais.
4. Define restrições do tipo **PROIBIDO** e **PERMITIDO**. A interface contém um atalho para selecionar pessoas que moram na mesma casa e criar proibições em ambos os sentidos.
5. Consulta o estado do sorteio e confirma o **sorteio definitivo**.
6. Recebe tokens individuais, pode gerar novos links e encaminhá-los pelo WhatsApp. A consulta completa dos resultados pelo organizador requer nova confirmação de senha na implementação atual.

2.2 Fluxo do participante

O código prevê um convite individual associado ao resultado. No primeiro acesso, o participante pode cadastrar uma senha; em acessos seguintes, entrar com evento, nome e senha. O serviço de participantes usa um JWT próprio, separado da sessão do organizador. A tela de resultado consulta somente os dados da identidade autenticada. **Esse fluxo precisa ser testado ponta a ponta no domínio do GitHub Pages**, conforme a revisão de rotas descrita no capítulo 15.

2.3 Regras centrais do sorteio

- **Mínimo de três participantes ativos.** Menos do que isso resulta em rejeição.
- **Sem autoatribuição.** Uma pessoa não tira a si mesma.
- **Destinatários únicos.** Cada pessoa é sorteada exatamente uma vez.
- **Sem duplas recíprocas.** Se A tira B, B não pode tirar A.
- **Restrições direcionais respeitadas.** Uma permissão/proibição parte de um participante para outro.
- **Impossibilidade explícita.** Quando as restrições não admitem solução, o algoritmo devolve `null` e a API comunica a falha, sem inventar um resultado.

A proibição recíproca é mais forte que a regra tradicional do amigo secreto. Em grupos de três, por exemplo, uma configuração pode exigir um ciclo de três pessoas; isso é permitido, mas pares de duas pessoas são vedados.

3. Arquitetura Full Stack



A interface **não acessa o banco diretamente**. Ela envia requisições HTTP à API. A API valida entrada, verifica identidade e autorização, executa as regras e persiste dados com o Prisma. O banco responde apenas ao backend. Isso reduz a exposição de credenciais e mantém as regras de sorteio em um local controlado.

Separação de responsabilidades:

Camada	Responsabilidade	Exemplo
Apresentação	Capturar ações e mostrar estados	`frontend/src/components/Painel.tsx`
Cliente HTTP	Reunir URL, cabeçalhos e tratamento de erros	`frontend/src/services/api.ts`
Controle HTTP	Receber requisições e devolver códigos/ status	`backend/src/routes/*.routes.ts`
Segurança	Validar tokens e verificar posse do evento	`middlewares/autenticacao.ts`, `permissaoEvento.ts`
Domínio	Encontrar um sorteio válido	`algorithms/sorteio.ts`
Persistência	Consultar e atualizar entidades	Prisma e `schema.prisma`
Infraestrutura	Publicação e variáveis de ambiente	Render, GitHub Actions e TiDB

4. Pré-requisitos e ferramentas

4.1 Instalações

Em um computador Windows, instale **Node.js**, **Git**, **Visual Studio Code** e **XAMPP**. O workflow atualmente usa **Node.js 24**; por isso, essa é uma escolha prática para reproduzir o ambiente de compilação. O projeto usa npm, TypeScript e comandos de terminal PowerShell. É útil ter o MySQL client para inspeções e migração de dados.

Verifique as instalações:

```
node --version
npm --version
git --version
```

Crie contas para **GitHub**, **Render** e **TiDB Cloud** somente quando for publicar a aplicação. A criação local não exige uma hospedagem paga.

4.2 Obter o código correto

A forma mais fiel de replicar o projeto existente é partir do repositório. Isso preserva os arquivos de rotas, componentes, esquema, testes e a configuração de publicação. O passo a passo abaixo ensina a compreender e operar o código; não pretende substituir centenas de linhas da implementação integral.

```
git clone https://github.com/Leonardocosta3/amigo-secreto.git
cd amigo-secreto
code .
```

A pasta raiz não contém um único programa executável: há **dois projetos npm**, `backend` e `frontend`, com instalações, comandos e variáveis diferentes.

4.3 Comandos essenciais do projeto

Comando	Executar onde	Efeito
`npm ci`	`backend` ou `frontend`	Instala dependências exatas do lockfile
`npm run dev`	`backend`	Inicia Express com recarga via `tsx watch`
`npm run dev`	`frontend`	Inicia Vite, normalmente em `localhost:5173`
`npm run build`	`backend`	Gera Prisma Client e compila TypeScript
`npm run build`	`frontend`	Verifica TypeScript e gera `dist` estático
`npm start`	`backend`	Executa `dist/server.js` compilado
`npx vitest run`	`backend`	Executa os testes automatizados existentes
`npx prisma migrate deploy`	`backend`	Aplica migrações já criadas no banco de destino

5. Estrutura de diretórios e função dos arquivos

A árvore abaixo destaca arquivos **efetivamente encontrados** no repositório. Algumas pastas contêm mais arquivos que os mostrados.

```
amigo-secreto/
|-- .github/workflows/deploy.yml    # GitHub Actions -> Pages
|-- .gitignore                       # Ignora segredos e builds
|-- LICENSE                          # Licenca MIT
|-- backend/
|   |-- package.json                 # Scripts e dependencias da API
|   |-- tsconfig.json                # Compilacao TypeScript
|   |-- prisma.config.ts             # Configuracao de migracoes
|   |-- prisma/schema.prisma         # Modelos de dados
|   |-- prisma/migrations/           # Historico das migracoes
|   |-- certs/ca.pem                 # Certificado CA publico, se presente
|   `-- src/
|       |-- server.ts                # Express, CORS e endpoints
|       |-- lib/prisma.ts            # Conexao Prisma + TLS
|       |-- algorithms/sorteio.ts    # Motor de sorteio
|       |-- algorithms/sorteio.test.ts
|       |-- middlewares/
|       |   |-- autenticacao.ts
|       |   `-- permissaoEvento.ts
|       `-- routes/                  # Rotas de cada dominio
`-- frontend/
    |-- package.json
    |-- vite.config.ts
    |-- index.html
    `-- src/
        |-- main.tsx
        |-- App.tsx
        |-- App.css / index.css
        |-- services/api.ts
        |-- services/participanteAuth.ts
        `-- components/
            |-- Login.tsx
            |-- Painel.tsx
            |-- Participantes.tsx
            |-- Restricoes.tsx
            |-- SorteioDefinitivo.tsx
            `-- Resultado.tsx
```

Como localizar rapidamente uma responsabilidade

Quando uma tela apresentar erro, comece pelo componente correspondente. Depois identifique o caminho enviado por `apiFetch` e procure essa rota em `backend/src/routes`. Por fim, descubra qual entidade Prisma é consultada e em que ponto uma autorização é exigida. Esse fluxo evita modificar arquivos aleatoriamente.

6. Construindo e compreendendo o backend

6.1 Node.js, Express e TypeScript

Node.js executa JavaScript no servidor; **Express 5** oferece métodos para definir rotas e middlewares; **TypeScript** adiciona checagem de tipos antes da compilação. No projeto, `tsx` executa TypeScript durante o desenvolvimento e `tsc` gera JavaScript para produção.

No `backend/package.json`, os scripts essenciais são:

```
{
  "scripts": {
    "dev": "tsx watch src/server.ts",
    "build": "prisma generate && tsc",
    "start": "node dist/server.js"
  }
}
```

Explicação: `dev` observa alterações e reinicia o servidor; `build` gera o cliente tipado para os modelos Prisma e compila o backend; `start` executa a saída em `dist`. Como o Render não deve depender de um processo de desenvolvimento permanente, ele usa `npm start` após o build.

O `backend/tsconfig.json` usa `rootDir: ./src`, `outDir: ./dist`, `target: ES2022`, resolução `NodeNext` e `strict: true`. Por isso, imports TypeScript podem terminar em `.js`: a referência permanece correta depois da emissão dos arquivos JavaScript. O `package.json` atual declara `"type": "commonjs"`; **não altere a estratégia de módulos isoladamente**, porque imports e compilação precisam permanecer coerentes.

6.2 Inicialização do servidor

Exemplo didático inspirado em `backend/src/server.ts`:

```
import express from "express";
import cors from "cors";

const app = express();
app.use(cors({ origin: ["http://localhost:5173"] }));
app.use(express.json());

app.get("/healthz", (_req, res) => {
  res.status(200).json({ status: "ok" });
});

const PORT = Number(process.env.PORT ?? 3001);
app.listen(PORT, "0.0.0.0");
```

`express()` cria a aplicação; `cors(...)` controla quais **origens de navegador** podem chamar a API; `express.json()` transforma corpos JSON em `req.body`; `app.get()` define uma rota HTTP GET; `res.json()` devolve JSON; `app.listen()` abre a porta. O host `0.0.0.0` é necessário para a API receber conexões externas no Render. **CORS não substitui autenticação.**

A versão real autoriza `localhost:5173`, `127.0.0.1:5173` e a origem do GitHub Pages, além de uma origem adicional por `FRONTEND_URL`. Ela também oferece `/`, `/healthz` e `/api/testar-banco`.

6.3 Organização em rotas

O servidor monta conjuntos de rotas por prefixo. Exemplo da ideia:

```
app.use("/api/auth", authRoutes);
app.use("/api/eventos", eventosRoutes);
app.use("/api/participantes", autenticar, participantesRoutes);
```

Uma rota declarada como `router.post("/login", ...)` no módulo `authRoutes` torna-se `POST /api/auth/login`. Usar `app.use(..., autenticar, ...)` protege todas as rotas daquele grupo. Os módulos de **eventos e sorteio** já aplicam `router.use(autenticar)` internamente; o fato de aparecerem sem middleware no `server.ts` não significa que sejam públicos.

6.4 Endpoints e contratos HTTP

Método e caminho	Objetivo	Acesso
`GET /healthz`	Verificar a execução da API	Público
`GET /api/testar-banco`	Testar a conexão ao SQL	Público atualmente; restringir em produção se necessário
`POST /api/usuarios`	Cadastrar organizador	Público na versão examinada
`POST /api/auth/login`	Entrar como organizador	Público
`GET /api/auth/me`	Validar a sessão atual	JWT de organizador
`GET/POST /api/eventos`	Listar e criar eventos	JWT + vínculo de usuário
`DELETE /api/eventos/:id`	Remover evento e dependências	JWT + dono do evento
`GET/POST /api/participantes`	Listar e incluir participantes	JWT + verificações
`GET/POST/DELETE /api/restricoes...`	Administrar restrições	JWT + verificações
`POST /api/sorteio/simular/:eventoId`	Simular o sorteio	JWT + dono do evento
`POST /api/sorteio/realizar/:eventoId`	Confirmar o sorteio	JWT + dono do evento
`GET /api/sorteio/status/:eventoId`	Consultar status	JWT + dono do evento
`POST /api/sorteio/resultados/:eventoId`	Resultado completo com senha	JWT + dono do evento
`GET /api/participante-auth/convite/:token`	Identificar convite	Token de convite
`POST /api/participante-auth/cadastrar`	Definir senha do participante	Convite válido
`POST /api/participante-auth/login`	Login do participante	Credenciais
`GET /api/participante-auth/resultado`	Resultado próprio	JWT de participante

Método e caminho	Objetivo	Acesso
`GET /api/resultado/:token`	Resultado por token, sem login	Público: revisar

Códigos frequentes: `200` sucesso; `201` criação; `400` entrada inválida; `401` autenticação ausente/incorreta; `404` recurso não encontrado ou inacessível; `409` conflito; `422` sorteio sem solução; `500` erro interno. O frontend deve tratar cada um sem apresentar detalhes sensíveis do servidor ao usuário.

7. Banco de dados e Prisma ORM

7.1 Por que usar um ORM

O Prisma permite representar entidades em um arquivo `.prisma` e realizar consultas como `prisma.evento.findMany(...)` em vez de montar SQL manual para cada operação. Também oferece tipagem no TypeScript, relações, índices e migrações versionadas.

O `schema.prisma` do projeto declara `provider = "mysql"`, mesmo quando o banco remoto é TiDB, porque a instância utilizada é compatível com o protocolo MySQL. O gerador `prisma-client` cria os arquivos em `backend/src/generated/prisma`.

7.2 Entidades implementadas

Modelo	Campos / regras de interesse	Relações
`Usuario`	`id`, `nome`, `email` único, `senhaHash`, `criadoEm`	Um usuário organiza eventos
`Evento`	Nome, descrição opcional, data, `usuarioId`, `sorteioRealizadoEm`	Organizador, participantes e resultados
`Participante`	Nome, e-mail/telefone opcionais, `ativo`, `eventoId`	Evento, acesso e resultado
`Restricao`	`eventoId`, `participanteId`, `destinatarioId`, `tipo`	Proibição/permissão direcional
`ResultadoSorteio`	Origem, destino, `tokenHash`, data	Registro final do sorteio
`ParticipanteAcesso`	Participante único, `senhaHash` e datas	Login individual

O enum `TipoRestricao` aceita **`PERMITIDO`** e **`PROIBIDO`**. Índices únicos ajudam a evitar duplicações: `Usuario.email`; composição de restrição; `ResultadoSorteio.participanteId`; `(eventoId, sorteadoId)`; `ResultadoSorteio.tokenHash`; e `ParticipanteAcesso.participanteId`.

7.3 Relações e integridade

Um organizador pode ter **vários eventos**. Cada evento tem **muitos participantes**. Um participante pode ter **um acesso** e uma atribuição de sorteador; cada resultado associa um participante a um destinatário. Restrições guardam pares dirigidos de IDs. Na exclusão de um evento, o backend apaga dependências **dentro de uma transação**, antes de remover o próprio evento.

Uma restrição de banco impede vários erros, mas **não substitui a regra de domínio**. Por exemplo, um índice único impede dois sorteados iguais para o mesmo evento, mas é o algoritmo que evita autoatribuição e trocas recíprocas.

7.4 Variáveis de ambiente: local e remoto

Mantenha o arquivo `backend/.env` para o banco local e `backend/.env.tidb` para comandos pontuais contra a nuvem. Ambos devem ficar fora do Git. O repositório já ignora `.env` e `.env.*`.

Exemplo didático local, nunca contendo senha real no repositório:

```
DB_HOST=127.0.0.1
DB_PORT=3306
DB_USER=amigo_app
DB_PASSWORD=SUBSTITUIR_POR_SENHA_LOCAL
```

```
DB_NAME=amigo_secreto
DB_SSL=false
MIGRATION_DATABASE_URL="mysql://USUARIO:SENHA@127.0.0.1:3306/amigo_secreto"
JWT_SECRET=GERAR_UMA_CHAVE_ALEATORIA_COM_32_OU_MAIS_CARACTERES
PARTICIPANTE_JWT_SECRET=GERAR_OUTRA_CHAVE_ALEATORIA_E_DIFERENTE
```

Na nuvem, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_SSL=true`, `DB_SSL_CA` e as chaves JWT são configurados no painel do Render. **Não coloque as credenciais do TiDB no frontend e não prefixe segredos com `VITE_`.** Variáveis `VITE_` são incorporadas aos arquivos públicos da interface.

Observação Prisma 7: `prisma.config.ts` usa `MIGRATION_DATABASE_URL` e, na versão examinada, também lê `MIGRATION_SHADOW_DATABASE_URL`. Se a variável shadow for exigida pela configuração na execução de um comando, forneça-a conforme o ambiente. O comando `migrate deploy` aplica migrações existentes e não deve ser confundido com `migrate dev`, que é usado para desenvolvimento e pode depender de banco shadow. Nunca use `migrate reset` no banco de produção.

7.5 Conexão segura com TiDB

O arquivo `backend/src/lib/prisma.ts` é o ponto central de acesso ao banco. Ele valida variáveis obrigatórias, configura `PrismaMariaDb` e cria `new PrismaClient({ adapter })`. Quando `DB_SSL=true`, lê um certificado de autoridade certificadora e configura `rejectUnauthorized: true`; isso permite verificar a identidade do servidor em uma conexão TLS.

Parâmetros existentes no projeto:

```
const adapter = new PrismaMariaDb({
  host: process.env.DB_HOST ?? "localhost",
  port: Number(process.env.DB_PORT ?? 3306),
  user: obterVariavel("DB_USER"),
  password: obterVariavel("DB_PASSWORD"),
  database: obterVariavel("DB_NAME"),
  connectionLimit: 5,
  connectTimeout: 10000,
  // ssl: objeto TLS quando DB_SSL=true
});
```

O host TiDB escolhido no processo pertence à região **sa-east-1**, na porta **4000**. Para replicar a instalação, cada pessoa deve obter **o próprio hostname, usuário, senha, certificado e política de IP** no console TiDB. Não reutilize contas ou senhas do projeto original.

7.6 Histórico de migrações

O histórico de execução incluiu quatro migrações Prisma aplicadas com sucesso ao TiDB:

1. `20261027122033_inicial`
2. `20261027131001_adicionar_restricoes`
3. `20261028133442_adicionar_resultados_sorteio`
4. `20261029115759_adicionar_acesso_participantes`

Em uma instalação nova, configure as variáveis e, **após confirmar que o endereço aponta para o banco correto**, aplique:

```
cd backend
npm ci
npx prisma generate
npx prisma migrate deploy
```

Se quiser selecionar explicitamente o ambiente TiDB, a ferramenta `dotenv-cli` já consta nas dependências de desenvolvimento:

```
npx dotenv -e .env.tidb -o -- npx prisma migrate deploy
```

A opção `-o` faz as variáveis do arquivo selecionado prevalecerem sobre outras já presentes no processo. Antes do comando, revise o nome do banco e mantenha um backup.

8. Autenticação, autorização e privacidade

8.1 Login do organizador

O arquivo `auth.routes.ts` busca o organizador pelo e-mail normalizado, compara a senha com `bcrypt.compare` e, se ela corresponder, gera um token com `jwt.sign`, usando `JWT_SECRET` e validade de **duas horas**.

```
const senhaValida = await bcrypt.compare(senha, usuario.senhaHash);
if (!senhaValida) return res.status(401).json({ erro: "Credenciais inválidas." });

const token = jwt.sign({}, segredo, {
  subject: String(usuario.id),
  expiresIn: "2h",
  algorithm: "HS256"
});
```

A senha **não é recuperada nem enviada de volta**; o hash é comparado. No cadastro de organizador (`usuarios.routes.ts`), o código usa `bcrypt.hash(senha, 12)` e grava `senhaHash`. O cadastro do organizador é público na versão analisada; antes de transformar a aplicação em serviço comercial, determine se qualquer usuário pode criar conta e imponha limites e verificações apropriadas.

8.2 Middleware Bearer

Em `middlewares/autenticacao.ts`, o servidor lê `Authorization: Bearer TOKEN`, verifica assinatura e expiração com a chave secreta, valida o identificador de usuário e o acrescenta ao objeto `req`. `next()` deixa a requisição continuar; respostas `401` interrompem chamadas não autenticadas. A variável `JWT_SECRET` deve ter pelo menos **32 caracteres** na regra atual.

8.3 Propriedade de evento

Em `permissaoEvento.ts`, a função `podeAdministrarEvento(usuarioId, eventoId)` consulta se existe um evento com **ambos** os valores. Não basta apresentar o ID de um evento válido; ele precisa pertencer ao organizador autenticado. Isso é crucial em aplicações com múltiplos organizadores para impedir leitura ou alteração de eventos alheios.

8.4 Acesso individual do participante

Há um fluxo **separado** em `participante-auth.routes.ts`:

- Um token de convite identifica o participante para o primeiro acesso.
- O participante registra senha de 8 a 72 caracteres, gravada em hash.
- O login posterior usa evento, nome e senha.
- O JWT de participante usa uma variável distinta, `PARTICIPANTE_JWT_SECRET`, e validade de **12 horas**.
- `GET /api/participante-auth/resultado` lê a identidade do JWT e retorna o próprio resultado.

Atenção operacional: configure `PARTICIPANTE_JWT_SECRET` no Render antes de testar o primeiro acesso. O fato de o login do organizador funcionar **não comprova** que o JWT do participante esteja corretamente configurado.

8.5 Tokens individuais de sorteio

O sistema gera 32 bytes aleatórios (`randomBytes(32)`) e transforma o valor em 64 caracteres hexadecimais. No banco, guarda **somente o hash SHA-256** do token. Isso reduz o impacto da exposição do banco: o token em texto puro é entregue ao organizador na geração, mas não pode ser recuperado diretamente do hash.

O endpoint de status devolve `token: null` para links antigos; por isso, a interface avisa para copiar os links quando são criados e permite gerar um novo link, invalidando o anterior. Esse é um comportamento intencional e importante para o suporte.

9. Motor de sorteio: explicação técnica

9.1 Por que não basta embaralhar nomes

Um simples `shuffle` pode produzir alguém tirando a si mesmo, duas pessoas tirando uma à outra ou uma atribuição proibida. Quanto maior o número de restrições, maior o risco de um embaralhamento ingênuo falhar. O sistema utiliza um problema de atribuição com restrições, resolvido por **busca recursiva e retrocesso**.

9.2 Preparação dos candidatos

Para cada participante, o algoritmo monta uma lista de destinatários permitidos. Ele remove a própria pessoa, aplica eventuais listas positivas `PERMITIDO` e exclui `PROIBIDO`. Se alguém ficar sem nenhum candidato, o resultado é imediatamente impossível.

```
const opcoes = ids.filter(destino => {
  if (destino === origem) return false;
  if (permitidos && !permitidos.has(destino)) return false;
  if (proibidos?.has(destino)) return false;
  return true;
});
```

Exemplo conceitual: se Ana e Bruno moram na mesma casa, criam-se `Ana → Bruno = PROIBIDO` e `Bruno → Ana = PROIBIDO`. A estrutura é direcional: impedir Ana de sortear Bruno não significa automaticamente o inverso, a menos que ambas as restrições sejam registradas.

9.3 Embaralhamento e ordem de busca

O embaralhamento usa `randomInt` do módulo `node:crypto`, aplicado como variante do Fisher-Yates. Os participantes com **menos alternativas** são priorizados na busca. Essa heurística procura detectar conflitos cedo, reduzindo tentativas infrutíferas em muitos cenários.

9.4 Retrocesso (backtracking)

O motor mantém um `Map` de atribuições e um `Set` de destinatários já usados. Para cada participante, tenta os candidatos disponíveis. Se uma escolha levar a um impasse mais adiante, **desfaz a atribuição** e experimenta outra possibilidade.

```
buscar(indice):
  se todos foram associados: sucesso
  para cada destinatario candidato:
    se destinatario ja foi usado: continuar
    se produz dupla reciproca: continuar
    associar e marcar como usado
    se buscar(proximo indice): sucesso
    desfazer associacao e marcacao
  retornar impossivel
```

O algoritmo explora combinações e, no pior caso, pode crescer muito com o número de participantes. Portanto, para eventos grandes ou restrições densas, é necessário medir tempo de resposta, eventualmente impor limites e avaliar uma modelagem de matching ou solver de restrições. **Não existe garantia universal de latência constante**.

9.5 Validação final

Antes de retornar, o código verifica novamente integridade dos IDs, unicidade de sorteadores/destinatários, proibições, permissões e inexistência de duplas recíprocas. Esse duplo controle é útil como defesa contra erros de lógica na busca.

9.6 Testes automatizados

O arquivo `sorteio.test.ts`, executado pelo Vitest, contém testes para sorteio válido, restrição `PROIBIDO`, restrição `PERMITIDO`, grupo de duas pessoas e conflito sem solução. Exemplos de casos adicionais recomendados: eventos com três pessoas; exclusões/inativação; participantes duplicados; restrições contraditórias; conjuntos com muitos participantes; múltiplas execuções aleatórias; e tempo máximo de execução em configurações patológicas.

```
cd backend
npx vitest run
```

10. Sorteio definitivo, transações e consistência

O sorteio definitivo precisa ser diferente da simulação. A **simulação** calcula um resultado e o devolve ao organizador, sem ser o compromisso final; o **definitivo** registra a decisão e precisa evitar duplicidade ou concorrência.

A implementação em `sorteio.routes.ts` busca os participantes ativos, restrições relevantes e executa o motor. Antes de persistir, valida que a atribuição cobre o conjunto esperado. A confirmação utiliza uma transação Prisma; o estado `sorteioRealizadoEm` atua como trava lógica. O mecanismo também realiza uma atualização condicional para detectar se outro pedido concluiu primeiro, retornando um conflito em vez de gravar dois sorteios.

Por que uma transação? Sem transação, uma falha no meio poderia deixar apenas parte dos resultados gravados. Com transação, as operações pertencem à mesma unidade lógica: ou se confirmam de forma consistente, ou são revertidas, respeitadas as garantias do banco.

Exemplo do princípio, não substituto da implementação real:

```
await prisma.$transaction(async tx => {
  // Verificar disponibilidade e marcar o evento atomica-
  // mente, condicionando a sorteioRealizadoEm = null.
  // Em seguida, persistir todos os resultados validos.
});
```

Para o operador, o sorteio definitivo é apresentado como permanente. Se o cliente pedir “refazer”, o procedimento comercial deve exigir política explícita: criar um novo evento ou implantar uma operação administrativa auditável, nunca apagar silenciosamente resultados já compartilhados.

11. Construindo e compreendendo o frontend

11.1 React, TypeScript e Vite

React monta a interface em componentes; **TypeScript** valida estruturas como `Usuario`, `Evento` e `Participante`; **Vite** entrega recarga rápida durante desenvolvimento e gera arquivos estáticos otimizados para hospedagem. O pacote `lucide-react` fornece ícones. A interface usa estilos em arquivos CSS próprios; a versão examinada não exige instalar Tailwind para funcionar.

11.2 Ponto de entrada

O `index.html` fornece a tag `<div id="root"></div>`. Em `src/main.tsx`, o React executa `createRoot(...).render(<App />)`, iniciando a aplicação. A tag `<title>` define o texto da aba do navegador; atualmente, no código consultado, está como **Amigo Secreto**. O cabeçalho de `Painel.tsx` utiliza o texto **Amigo Secreto 2026**; são configurações independentes.

11.3 Estado e navegação em `App.tsx`

O componente principal mantém três estados: `usuario`, `eventoId` e `verificando`. Ao abrir o sistema, consulta se existe um token de organizador e, se existir, chama `GET /api/auth/me` para restaurar a sessão. Se não houver sessão, mostra `Login`; se houver evento selecionado, mostra `Restricoes`; caso contrário, mostra `Painel`.

```
if (verificando) return <div>Verificando sessão...</div>;
if (!usuario) return <Login onLogin={setUsuario} />;
if (eventoId !== null) {
  return <Restricoes eventoId={eventoId} onVoltar={() => setEventoId(null)} />;
}
return <Painel usuario={usuario} onSair={sair} />;
```

Conceitos: `useState` representa dados que mudam na interface; `useEffect` executa efeitos de sincronização, como carregar dados; props (`eventoId`, `onVoltar`) passam informações e callbacks entre componentes; renderização condicional escolhe a tela.

11.4 Cliente HTTP centralizado

O arquivo `services/api.ts` escolhe a URL da API por `import.meta.env.VITE_API_URL` e usa `http://localhost:3001` quando a variável não existe. O token do organizador fica no `sessionStorage` e é inserido em `Authorization: Bearer ...` antes de `fetch()`. Respostas HTTP com erro geram exceções apresentáveis nos componentes.

```
const API_URL = import.meta.env.VITE_API_URL || "http://localhost:3001";
const resposta = await fetch(`${API_URL}/api/eventos`, {
  headers: { Authorization: `Bearer ${token}` }
});
```

Usar um serviço central evita espalhar URLs e cabeçalhos por dezenas de componentes. **`sessionStorage` não elimina o risco de XSS**; para amadurecer o produto, avalie uma política de sessão mais robusta, CSP, saneamento de entradas e revisão de dependências.

11.5 Principais componentes

Componente	O que faz	Trecho conceitual de lógica
`Login.tsx`	Formulário de e-mail/senha e token	`POST /api/auth/login`
`Painel.tsx`	Lista, cria e remove eventos	`GET/POST/DELETE /api/eventos`
`Participantes.tsx`	Cadastra e exclui participantes	`POST /api/participantes`
`Restricoes.tsx`	Seleciona pessoas e cria proibições	`POST /api/restricoes`
`SorteioDefinitivo.tsx`	Confirma sorteio, gera links, resultados	`POST /api/sorteio/realizar/:id`
`Resultado.tsx`	Primeiro acesso, login e resultado próprio	`/api/participante-auth/...`

Em `Painel.tsx`, o formulário verifica `nomeEvento`, chama a API e recarrega a lista. Para excluir, o usuário digita o nome do evento como confirmação. Em `Participantes.tsx`, e-mail e telefone são opcionais. Em `Restricoes.tsx`, um `Set` evita adicionar pares duplicados no lado do cliente, mas a prevenção robusta também depende do banco e da API.

11.6 Restrição “pessoas da mesma casa”

O atalho seleciona pelo menos duas pessoas. Para cada combinação de origem e destino distintos, cria uma restrição `PROIBIDO`. Se forem selecionadas três pessoas A, B e C, haverá seis pares dirigidos: A→B, A→C, B→A, B→C, C→A e C→B. A interface envia os pares à API e atualiza a lista após salvar. Se uma requisição falhar no meio do conjunto, o frontend recarrega os dados para refletir alterações parciais; uma melhoria futura é **uma rota de criação em lote transacional**.

11.7 Compartilhamento pelo WhatsApp

A interface formata o telefone, monta uma mensagem com o link exclusivo e abre `https://wa.me/...`. Isso é **compartilhamento por link**, não uma integração oficial automatizada com a API de mensagens do WhatsApp. Não confunda o recurso com envio automático em massa, que tem regras, custos e integrações próprias.

12. Execução local passo a passo

12.1 Criar banco e usuário local

No XAMPP, inicie o MySQL/MariaDB e abra o phpMyAdmin. Crie um banco `amigo_secreto` com codificação Unicode. Recomenda-se criar um usuário dedicado com permissões necessárias ao desenvolvimento, em vez de usar `root` sem senha.

Exemplo SQL ilustrativo (executar com um administrador do banco):

```
CREATE DATABASE amigo_secreto
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
CREATE USER 'amigo_app'@'localhost'
  IDENTIFIED BY 'SUBSTITUA_POR_SENHA_FORTE';
GRANT ALL PRIVILEGES ON amigo_secreto.*
  TO 'amigo_app'@'localhost';
```

O contexto XAMPP pode ter uma conta `root` configurada de maneira diferente; ajuste o comando às permissões reais. **Não use a senha de exemplo em um ambiente acessível pela internet**.

12.2 Preparar o backend

```
cd amigo-secreto\backend
npm ci
# Crie .env local com DB_*, JWT_SECRET e MIGRATION_DATABASE_URL
npx prisma generate
npx prisma migrate deploy
npm run dev
```

Teste `http://localhost:3001/healthz` e `http://localhost:3001/api/testar-banco`. O primeiro atesta que o processo está respondendo; o segundo executa uma consulta real ao banco. Se o login falhar, lembre-se de que instalar o schema **não cria automaticamente** um organizador: faça um cadastro autorizado pela rota de usuários ou execute um seed controlado sem guardar senhas em claro.

12.3 Preparar o frontend

Em **outro terminal**, na pasta do frontend:

```
cd amigo-secreto\frontend
npm ci
# Para desenvolvimento, a API local e o padrao:
# http://localhost:3001
npm run dev
```

Abra `http://localhost:5173/amigo-secreto/` se o Vite servir com a base configurada; use o endereço exato exibido no terminal. Para alterar a API local, crie `frontend/.env.local` com `VITE_API_URL=http://localhost:3001` (arquivo ignorado pelo Git). Reinicie o Vite depois de alterar variáveis de ambiente.

12.4 Sequência de teste local

1. Confirmar `healthz` e consulta ao banco.
2. Cadastrar ou disponibilizar um organizador de teste.
3. Autenticar e validar `/api/auth/me`.

4. Criar evento e listar somente eventos do usuário.
5. Cadastrar cinco participantes; excluir um e verificar atualização da interface.
6. Criar uma restrição de casa; confirmar que o sistema impede pares proibidos.
7. Executar simulação; validar autoatribuição, unicidade e pares recíprocos.
8. Confirmar sorteio definitivo em evento descartável.
9. Testar link, registro de senha e acesso do participante.
10. Reiniciar backend e navegador; confirmar persistência e restauração correta.

13. Versionamento e GitHub

O Git permite registrar cada avanço com histórico de alterações. O GitHub mantém cópia remota e serve de origem para Render e GitHub Actions. Comandos usados no processo:

```
git status
git add .
git commit -m "Descrever uma alteracao concreta"
git push origin main
```

Boas práticas: faça commits pequenos; revise `git status` e `git diff --cached` antes de enviar; mantenha `.env`, `.env.tidb`, senhas, arquivos compilados `dist` e `node_modules` fora do Git; mantenha `package-lock.json` versionado para instalações reproduzíveis. Certificados de CA públicos podem ser versionados quando não incluem chave privada, mas confirme exatamente o conteúdo.

Problema que ocorreu no projeto: havia artefatos de compilação `backend/dist` rastreados. A correção foi ajustar `.gitignore` e remover os artefatos do índice com `git rm -r --cached backend/dist`, sem apagar o código-fonte local. Isso impede diferenças de plataforma e envios desnecessários.

14. Migrando de XAMPP para TiDB Cloud

14.1 Por que migrar

Um banco XAMPP funciona enquanto o computador está ligado e acessível localmente; uma aplicação hospedada no Render exige um banco alcançável a partir da internet, com conexão segura e limites de uso definidos. Foi escolhido **TiDB Cloud Starter**, modalidade compatível com MySQL, mantendo o Prisma e as estruturas principais.

14.2 Procedimento de replicação

1. Criar uma instância TiDB Cloud Starter em modo MySQL compatível e obter as informações de conexão.
2. Criar o banco lógico `amigo_secreto` (ou nome escolhido).
3. Baixar o certificado CA indicado pelo provedor e manter a validação TLS ativa.
4. Criar `backend/.env.tidb` com `DB_HOST`, `DB_PORT=4000`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_SSL=true`, `DB_SSL_CA=certs/ca.pem` e a URL de migração apropriada.
5. Aplicar `npx dotenv -e .env.tidb -o -- npx prisma migrate deploy` após conferir o destino.
6. Transferir somente os dados desejados do ambiente local, preservando integridade referencial e hashes de senha.
7. Executar uma consulta de teste com o adaptador Prisma.
8. Testar o backend local apontando ao TiDB **sem alterar o `.env` destinado ao XAMPP**.

14.3 Transferência de dados

O procedimento histórico migrou a conta do organizador e depois testou o login remoto. Para uma cópia completa, **não basta exportar uma tabela sem considerar relações**: há eventos, participantes, restrições, resultados e acessos. Planeje exportação e importação coerentes, com backup, controle de IDs, integridade de chaves estrangeiras e período de congelamento das gravações.

Em Windows/PowerShell, uma exportação redirecionada pode ter codificação inesperada, e o import pode falhar ou corromper acentos. No processo original foi necessário corrigir a saída para **UTF-8** antes da importação. Sempre confira nomes com acentos e datas após restaurar.

Importante: não armazene dump real de usuários nem de senhas hash em repositório público. Um hash ainda é dado de autenticação sensível.

15. Publicação na internet: a configuração utilizada

15.1 Backend no Render

Criar um **Web Service** e conectar o repositório GitHub. Na configuração do serviço, usar o diretório raiz `backend`, branch `main`, ambiente Node.js e plano selecionado. O serviço foi publicado em:

https://amigo-secreto-api-xdxv.onrender.com

Campo	Valor aplicado
Root directory	`backend`
Branch	`main`
Build Command	`npm ci --include=dev && chmod +x node_modules/.bin/prisma && npm run build`
Start Command	`npm start`
Health check	`/healthz`
Porta	Variável `PORT` fornecida pelo Render, com fallback local 3001

Variáveis de ambiente importantes: `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_SSL=true`, `DB_SSL_CA`, `JWT_SECRET`, `PARTICIPANTE_JWT_SECRET`, `FRONTEND_URL`, `NODE_ENV=production`. No processo original, o login do organizador foi validado; a chave específica do participante deve ser checada separadamente.

O endpoint `/healthz` retorna estado do servidor, sem consultar o banco; `/api/testar-banco` verifica a conexão SQL. Para a documentação do cliente, guarde os links e instruções de resposta esperada.

15.2 Problemas reais de build no Render

Erro 1: `sh: 1: prisma: Permission denied`. O arquivo executável do Prisma não tinha a permissão necessária no ambiente de compilação Linux. Foi aplicado o comando com `chmod +x node_modules/.bin/prisma` e limpeza de cache do build.

Erro 2: `Cannot find module 'vitest'` e declarações `@types/express`, `@types/jsonwebtoken`, `@types/bcrypt`, `@types/cors` ausentes. O build estava ocorrendo sem todas as dependências de desenvolvimento, apesar de estarem presentes no computador. A solução foi **`npm ci --include=dev`**, mantendo `npm run build`.

Princípio: projetos TypeScript frequentemente precisam de pacotes `devDependencies` para **compilar**, mesmo quando o servidor final roda em produção. Depois do build, o processo de execução usa JavaScript gerado em `dist`.

15.3 Frontend no GitHub Pages

O arquivo `frontend/vite.config.ts` usa `base: "/amigo-secreto/"`. Isso é necessário porque o site não está publicado no domínio raiz, mas em um caminho de repositório.

O workflow `.github/workflows/deploy.yml` é acionado por `push` em `main` ou manualmente por `workflow_dispatch`. Ele usa Node.js 24, instala dependências com `npm ci`, compila em `frontend`, publica `frontend/dist` por `actions/upload-pages-artifact` e `actions/deploy-pages`.

O valor público `VITE_API_URL` é configurado em **GitHub → Settings → Secrets and variables → Actions → Variables**, com:

```
VITE_API_URL=https://amigo-secreto-api-xdxv.onrender.com
```

Essa variável não é segredo: o navegador precisa conhecer o endereço público da API. Após alterá-la, reexecute o workflow; a URL é incorporada durante o build, não modificada dinamicamente nos arquivos já publicados.

Site publicado: https://leonardocosta3.github.io/amigo-secreto/

15.4 Fluxo de atualização

1. Modificar arquivo no VS Code.
2. Executar `npm run build` localmente e, quando pertinente, `npx vitest run`.
3. Revisar `git diff` e enviar `git push origin main`.
4. Acompanhar o deploy do Render quando houver mudanças no backend.
5. Acompanhar GitHub Actions quando houver mudanças no frontend.
6. Fazer uma verificação real no navegador, usando `Ctrl + Shift + R` se necessário.
7. Confirmar funcionalidade, não apenas o indicador verde do workflow.

16. Pontos críticos encontrados na revisão do código

> **Atenção antes da comercialização:** os itens desta seção não impedem o uso como prova de conceito, mas precisam de revisão específica antes de vender o acesso a um público mais amplo. Foram identificados pela leitura do código publicado; não são alegações de que um ataque ou vazamento tenha ocorrido.

16.1 Links individuais e GitHub Pages

Situação observada: `SorteioDefinitivo.tsx` monta um endereço com `window.location.origin + "/resultado/" + token`. Em GitHub Pages, isso gera algo semelhante a `https://leonardocosta3.github.io/resultado/TOKEN`, **sem o segmento `/amigo-secreto/`**. Além disso, o GitHub Pages hospeda arquivos estáticos: acessar diretamente um caminho de SPA sem arquivo correspondente pode retornar HTTP 404, mesmo com o prefixo correto. `App.tsx` também procura o token em `window.location.pathname` a partir de `/resultado/`.

Correção sugerida mais simples: usar roteamento por **fragmento/hash** para links individuais, que não depende de fallback do servidor. Adaptar em conjunto a criação dos links e a leitura em `App.tsx`:

```
// Em SorteioDefinitivo.tsx (exemplo de adaptacao)
function obterUrl(token: string) {
  const base = import.meta.env.BASE_URL; // /amigo-secreto/
  return `${window.location.origin}${base}#/resultado/${token}`;
}

// Em App.tsx, identificar pela hash (exemplo)
const tokenResultado = window.location.hash.match(
  /^#\/resultado\/([a-fA-F0-9]{64})\/?$/
)?.[1];
```

Depois, publicar novamente e **testar o link em uma janela anônima**, com o navegador fechado entre tentativas. Essa é uma **proposta de correção**, não uma mudança já aplicada. Alternativa: usar roteador com configuração de fallback/redirect em uma hospedagem que suporte SPA.

16.2 Endpoint público de resultado por token

O backend mantém `GET /api/resultado/:token`, que entrega o resultado individual a quem tiver o token válido, **sem exigir o JWT do participante**. Ao mesmo tempo, `participante-auth.routes.ts` oferece um fluxo mais restrito com senha e sessão. Isso representa dois modelos de sigilo coexistindo: **posse do link** versus **autenticação com senha**.

Se a proposta comercial exigir que o participante sempre forneça senha, é preciso **desativar ou reestruturar o endpoint público** e verificar todos os caminhos que permitem revelar resultados. Um link tokenizado é um segredo de acesso: não deve aparecer em relatórios de analytics, logs públicos, capturas ou mensagens encaminhadas sem controle. Reemitir link deve invalidar o anterior.

16.3 Outras recomendações de endurecimento

- Verificar `PARTICIPANTE_JWT_SECRET` e manter segredo diferente do `JWT_SECRET`.
- Revisar o endpoint público de cadastro de organizadores: limitar criação, validar e-mail, prevenir abuso e automatizar recuperação segura de acesso.

- Avaliar proteção de `/api/testar-banco` em produção: ele expõe estado operacional, mesmo sem divulgar credenciais.
- Revisar limitação de tentativas também no login de organizador e em endpoints que geram trabalho intenso.
- Configurar política de cookies/sessão e **Content Security Policy** adequada ao frontend; mitigar XSS.
- Implementar backups com **teste de restauração** e política de retenção.
- Revisar CORS, autenticação, autorização de cada rota e limites de tamanho do corpo JSON.
- Auditar dependências e implementar monitoração, alertas e tratamento de falhas.
- Testar duplicidade de nomes de participantes no mesmo evento, pois o login individual usa evento + nome + senha e detecta colisões.
- Incluir política de privacidade e práticas compatíveis com a **LGPD** quando coletar e-mail, telefone e nome de pessoas.

17. Testes recomendados e critérios de aceite

Um indicador `Success` no GitHub Actions valida uma publicação, **não todo o produto**. A aceitação de produção deve considerar cenários positivos, falhas e isolamento entre contas.

Teste	Procedimento	Resultado esperado
API viva	Abrir `/healthz`	HTTP 200 e status ok
Banco	Abrir `/api/testar-banco`	HTTP 200 quando TiDB acessível
Organizador	Login correto	JWT e painel do próprio usuário
Senha errada	Login inválido	HTTP 401 sem expor hash
Isolamento	Usuário B tenta evento A	Acesso negado/404
Cadastro	Criar evento e participantes	Persistência após recarregar
Restrição	Cadastrar duas pessoas da casa	Nenhum sorteio viola o par
Regras	Somente 2 participantes	Sorteio recusado
Regras	Restrições incompatíveis	HTTP 422 / sem resultado parcial
Sorteio definitivo	Dois pedidos quase simultâneos	Um único resultado persistido
Convite	Link aberto anonimamente	Tela apropriada, sem 404 do Pages
Primeiro acesso	Cadastrar senha do participante	Hash persistido e JWT próprio
Resultado	Login do participante A	Somente resultado de A
Renovação	Gerar novo link	Token anterior deixa de valer
Suspensão	Acessar após inatividade	App informa espera e recupera
Backup	Restaurar cópia em ambiente isolado	Dados e relações consistentes

Instrumentação e depuração

No navegador, `F12 → Network` mostra URL, método, status e tempo de resposta. `F12 → Console` mostra erros JavaScript. No Render, verifique **Logs** e status do deploy. No TiDB, monitore conexões e consumo de recursos. Em caso de falha, registre horário, ação, endpoint, código HTTP e identificador do evento **sem copiar tokens nem senhas**.

18. Solução de problemas recorrentes

Sintoma	Causa provável	Verificação / ação
`prisma: Permission denied`	Executável sem permissão Linux/ cache	`chmod +x ...`, limpar cache e verificar dependências rastreadas
Falta `vitest` e `@types`	Dependências `dev` omitidas	`npm ci --include=dev` no build
`P1000` Prisma	Credenciais de banco incorretas	Conferir usuário/senha e ambiente selecionado
`P1010`, `P3014`	Privilégios insuficientes/shadow DB	Rever permissões e comandos Prisma apropriados
`Pool ... limit=5`	Exaustão de conexões	Inspecionar processos, paralelismo e sessões; não criar clientes por requisição
Handshake / timeout	Banco desligado, rede, TLS, limite	Conferir XAMPP ou TiDB, porta e CA
`Access denied` MySQL	Usuário/host/senha/permissão	Testar cliente SQL com a mesma conta
CORS no navegador	Origem frontend não permitida	Atualizar `FRONTEND_URL`/lista no backend
Frontend chama localhost	`VITE_API_URL` não aplicado	Atualizar GitHub Variable e republicar
Página branca no Pages	`base` ou assets incorretos	Conferir `vite.config.ts` e Console/Network
Link individual dá 404	Caminho SPA sem fallback	Corrigir hash ou configurar fallback (cap. 16)
JWT inválido	Chave diferente, token expirado	Checar segredos e relógio; renovar login
Sorteio 422	Regras não admitem solução	Revisar restrições e pessoas ativas
Sorteio 409	Sorteio já confirmado / concorrência	Consultar status antes de refazer
Acentos quebrados	Exportação/importação com encoding distinto	Usar UTF-8 e verificar collation
`nothing to commit`	Nenhum arquivo versionado mudou	Conferir `git status` e `git diff`

Regra de diagnóstico: verifique uma camada por vez: navegador → API → Prisma → banco. Assim, erros de infraestrutura não são confundidos com defeitos do algoritmo.

19. Operação, backups e custos da hospedagem

19.1 Ambiente gratuito é bom para validação

A combinação GitHub Pages + Render Free + TiDB Starter permitiu **demonstrar** o sistema sem manter computador ligado. Entretanto, não equivale a SLA, disponibilidade contínua, backup contratado ou operação comercial garantida.

O Render Free pode **suspender o serviço após 15 minutos sem tráfego** e demorar a reiniciar; tem limites de horas e uso. O próprio Render informa que suas instâncias gratuitas **não devem ser usadas para aplicações de produção**. Por isso, para clientes pagantes, planeje infraestrutura apropriada com monitoramento, política de recuperação, orçamento e eventual upgrade.

19.2 Restrições comerciais do GitHub Pages

A documentação oficial do GitHub afirma que **GitHub Pages não se destina e não pode ser usado como hospedagem gratuita para operar um negócio online, e-commerce ou software comercial como serviço (SaaS)**. Portanto, embora seja adequado ao portfólio e à demonstração técnica, **não se recomenda manter o produto comercial pago no GitHub Pages**. Ao vender o acesso, mova o frontend para uma hospedagem cuja política aceite uso comercial, como um serviço estático apropriado, e valide os termos vigentes.

19.3 Checklist operacional

- Manter contas e domínios sob controle documentado; ativar autenticação multifator.
- Guardar segredos em cofre apropriado, não em mensagens ou planilhas sem proteção.
- Fazer backup do SQL e testar restauração num banco separado antes de confiar no processo.
- Definir frequência de backup compatível com a perda máxima aceitável.
- Monitorar disponibilidade, erros HTTP, conexão ao TiDB e limites de planos.
- Registrar mudanças relevantes e manter procedimento de rollback.
- Definir quem atende usuários, horário de suporte e plano de incidente.
- Revisar renovação de domínios e cobranças de fornecedores.
- Revalidar fluxos de participante após qualquer alteração de roteamento ou autenticação.

20. Comercialização: o que é possível vender?

O produto pode ser comercializado como **serviço de organização**, **acesso a uma plataforma**, **implantação personalizada**, **licenciamento de componentes** ou **serviços de desenvolvimento e manutenção**. Esses modelos têm responsabilidades técnicas e jurídicas diferentes.

20.1 Modelo A — Operação por evento

Você recebe a lista de participantes, cria o evento, configura regras e entrega convites/links. O cliente compra **a execução e o acompanhamento**, não a propriedade do código. É o modelo mais simples para testar aceitação, especialmente em datas comemorativas.

Entregáveis sugeridos: configuração do evento, até uma quantidade contratada de participantes, validação prévia, sorteio definitivo, orientação para distribuição e suporte por uma janela definida. Deixe explícito que a confirmação definitiva não pode ser refeita sem regra contratual.

Exemplo de preço ilustrativo, não cotação de mercado: R$ 29 a R$ 99 por evento pequeno, variando com número de participantes, suporte e personalização.

20.2 Modelo B — Assinatura SaaS

Vários organizadores entram na mesma plataforma, cada qual vendo apenas os seus eventos. Uma assinatura pode liberar cotas de eventos/participantes, personalização, suporte e relatórios. Exige mais do que uma tela de pagamento: cobrança recorrente, planos, limites, cancelamentos, dados do titular, suporte, monitoramento e capacidade.

Exemplo de preço ilustrativo: plano individual de R$ 19 a R$ 39/mês e plano organizacional de R$ 79 a R$ 199/mês, condicionados a teste de disposição a pagar e aos custos reais. **Para uso comercial, migrar do GitHub Pages e do Render Free para serviços compatíveis com produção.**

20.3 Modelo C — Implantação sob marca do cliente (white label)

Personalizar identidade visual, título, ícone, domínio, textos e fluxos para uma escola, empresa, associação ou agência de eventos. Pode ser vendido como **taxa inicial + manutenção**. É importante decidir se cada cliente terá banco isolado ou se haverá multitenancy segura. Para poucas implantações, um ambiente separado por cliente facilita isolamento; para escalar, exige automação e arquitetura mais madura.

Exemplo ilustrativo: R$ 800 a R$ 3.000 por implantação simples, mais mensalidade de suporte/hospedagem acordada. Customizações, integrações e segurança ampliada podem elevar substancialmente o esforço.

20.4 Modelo D — Licenciar apenas uma parte

Componentes possíveis: **motor de sorteio com restrições**, **módulo de cadastro**, **interface React**, **módulo de acesso individual** ou **implantação da API**. Pode ser fornecido como biblioteca, API privada, consultoria de integração ou projeto derivado. É necessário documentar interfaces, limites, testes e suporte; disponibilizar uma API não cria automaticamente um produto pronto para terceiros.

Exemplo de entrega parcial: fornecer o algoritmo de sorteio, especificação da estrutura de entrada/saída, testes e treinamento para integrar ao sistema do cliente. O pagamento remunera implementação, parametrização e suporte — não necessariamente exclusividade do código.

20.5 Modelo E — Serviços profissionais

Uma opção comercial imediata é vender **instalação, treinamento, configuração, personalização, correção de bugs, evolução de funcionalidades e suporte**, mesmo que o software-base seja de código aberto. O cliente paga pelo trabalho e pelo nível de serviço. Isso pode ser mais viável antes de investir em cobrança automática e operação SaaS 24/7.

21. Licença MIT, propriedade intelectual e contratos

O repositório público analisado contém um arquivo `LICENSE` **com licença MIT**. Esse é um ponto comercial central: a MIT permite que terceiros **usem, copiem, modifiquem, distribuam, sublicenciem e vendam cópias** do software, desde que cumpram as condições de preservação do aviso de direitos autorais e da licença.

Isso **não impede** que você cobre por personalização, suporte, hospedagem, instalação ou desenvolvimento adicional. Porém, **não é apropriado prometer exclusividade sobre o código-base MIT já publicado**. Uma mudança posterior de licença não retira automaticamente permissões concedidas a cópias previamente obtidas sob MIT. Marcas e ativos de terceiros também exigem análise separada.

Antes de vender, defina contratualmente:

Tema	Decisão que precisa estar escrita
Objeto	Prestação de serviço, licença, manutenção ou entrega de código
Propriedade	Código existente, código novo, personalizações e direito de reutilização
Hospedagem	Quem paga fornecedores e administra contas/domínios
Segurança	Backups, segregação de dados, incidentes e acessos
Suporte	Canais, horários, prazo de resposta e exclusões
Evolução	Funcionalidades incluídas e mudanças cobradas separadamente
Cancelamento	Exportação/devolução e eliminação de dados
Pagamento	Entrada, recorrência, prazo, inadimplência e reajuste
Privacidade	Responsáveis, finalidade, retenção, transparência e LGPD

Esses itens são recomendações contratuais gerais, **não substituem avaliação jurídica e contábil** para a operação escolhida.

22. Estratégia comercial sugerida para começar

Fase 1 — Validação técnica (prioridade imediata). Corrigir os links dos participantes; reconciliar o endpoint público de resultado com a regra de senha; confirmar a chave JWT de participantes; executar checklist de isolamento e teste real com 5 a 10 usuários convidados. Preparar backup e política de atendimento.

Fase 2 — Oferta inicial. Vender **operação por evento** ou **implantação assistida**, com escopo fixo, suporte curto e pagamento por serviço. Evita que uma aplicação de teste seja divulgada como um SaaS com disponibilidade garantida. Validar o que clientes realmente valorizam: privacidade, proibições entre familiares, praticidade de WhatsApp, suporte e visual com a marca deles.

Fase 3 — Produto recorrente. Hospedar o frontend numa opção que aceite SaaS comercial; contratar backend adequado a produção; desenvolver onboarding de organizadores, recuperação de conta, planos/limites e pagamento. Formalizar termos de uso, privacidade, logs e relatórios de uso.

Fase 4 — Escala. Automatizar implantação, backups, testes e observabilidade; avaliar filas, rate limiting distribuído, otimização do motor para eventos grandes, isolamento de clientes e maior cobertura de testes.

22.1 Conta simples para precificação

Preço não deve depender apenas do custo mensal de nuvem: inclua suporte, operação, marketing, impostos, tempo de implantação, meios de pagamento e risco.

Cenário ilustrativo	Receita bruta	O que ainda precisa ser abatido
20 eventos × R$ 39	R$ 780	Tempo de atendimento, taxas, impostos e infraestrutura
40 assinantes × R$ 29/mês	R$ 1.160/mês	Hospedagem paga, suporte, churn, tributos e pagamentos
3 implantações × R$ 1.200	R$ 3.600	Horas de entrega, configuração, manutenção e impostos

Os valores acima são **hipóteses para análise**, não pesquisa de preços de mercado ou promessa de lucro. Uma planilha comercial deve calcular margem por hora e custo de atendimento por cliente.

23. Roadmap de evolução priorizado

Prioridade	Melhoria	Benefício e critério de sucesso
P0	Corrigir links individuais no Pages	Convite abre diretamente, inclusive em navegador anônimo
P0	Unificar política de sigilo do resultado	Nenhum caminho contorna a autenticação exigida
P0	Validar `PARTICIPANTE_JWT_SECRET`	Cadastro, login e resultado próprio funcionando em produção
P0	Backup e restauração testados	Recuperação demonstrada com cópia real
P1	Limitar login/cadastro e auditar acesso	Reduzir abuso e tentativas automatizadas
P1	Mensagem de “servidor iniciando”	Experiência clara durante cold start do Render
P1	Testes de API, integração e E2E	Regressões detectadas antes do deploy
P1	Criar restrições em lote com transação	Evitar cadastros parciais em redes instáveis
P1	Revisar fluxo de nomes repetidos	Login individual não depende de nome ambíguo
P2	Hospedagem comercial e domínio próprio	Operação compatível com modelo de venda escolhido
P2	Painel de cobrança/assinatura	Monetização recorrente controlada
P2	Auditoria e observabilidade	Rastreabilidade sem expor segredos
P3	Escala do algoritmo e filas	Resposta estável em eventos grandes
P3	White label configurável por cliente	Implantação replicável sem editar código manualmente

24. Procedimento de entrega para outra pessoa

Para que um terceiro replique o projeto com menos risco, entregue **o repositório, este manual, um checklist de variáveis sem segredos e um ambiente de teste isolado**. A pessoa deve conseguir completar a seguinte sequência sem acessar suas contas:

1. Clonar o repositório público e instalar Node.js 24, Git e VS Code.
2. Iniciar MySQL/MariaDB local, criar usuário dedicado e banco vazio.
3. Criar `.env` local, rodar migrações e iniciar a API.
4. Rodar o frontend local e criar um organizador de teste.
5. Validar cadastro, restrições, sorteio e acesso individual.
6. Criar **contas próprias** no GitHub, Render e TiDB Cloud.
7. Configurar a base remota, TLS e migrações sem importar dados pessoais do projeto original.
8. Publicar backend com variáveis protegidas e testar SQL.
9. Publicar frontend configurando `VITE_API_URL` e o caminho base do repositório escolhido.
10. Validar links individuais e autorizações, fazer backup e documentar responsáveis.

Critério final: não basta carregar a página. A entrega é considerada completa quando o terceiro consegue criar um evento, adicionar participantes, impedir um par proibido, concluir um sorteio válido, acessar um resultado individual da forma definida pelo produto e recuperar os dados após reiniciar serviços.

Apêndice A — Referência rápida de comandos

```
# Desenvolvimento local (terminal 1)
cd amigo-secreto\backend
npm ci
npx prisma generate
npx prisma migrate deploy
npm run dev

# Interface local (terminal 2)
cd amigo-secreto\frontend
npm ci
npm run dev

# Verificacoes e compilacoes
cd amigo-secreto\backend
npx vitest run
npm run build
cd ..\frontend
npm run build

# Versionamento (na raiz do projeto)
cd ..
git status
git add .
git commit -m "Descrever alteracao"
git push origin main
```

Comandos destrutivos não incluídos no fluxo de replicação: `prisma migrate reset`, `DROP DATABASE`, `TRUNCATE`, exclusão forçada de dados ou substituição de credenciais sem backup. São inadequados para uso rotineiro num ambiente com participantes reais.

Apêndice B — Glossário essencial

Termo	Significado prático
API REST	Endpoints HTTP que recebem dados e devolvem JSON
Backend	Código executado no servidor
Frontend	Interface executada no navegador
ORM	Biblioteca que mapeia entidades do código para tabelas SQL
Migração	Alteração de estrutura do banco registrada e versionada
JWT	Token assinado que representa sessão/identidade
Hash de senha	Derivação irreversível usada para comparar senhas sem guardá-las em claro
CORS	Regra de navegador que controla origens autorizadas a ler respostas
TLS	Proteção criptográfica e validação da conexão de rede
Backtracking	Busca que desfaz escolhas para tentar outras combinações
Transação	Grupo de alterações ao banco tratado como uma unidade
CI/CD	Automatização de testes, compilação e publicação
White label	Solução adaptada visualmente à marca de um cliente
SaaS	Software acessado como serviço, geralmente por assinatura
Cold start	Tempo de reinicialização após serviço ficar suspenso
LGPD	Lei Geral de Proteção de Dados Pessoais do Brasil

Fontes e referências

Código e configuração do projeto (consulta em 01/11/2026)

- Repositório público – Amigo Secreto
- Backend: package.json
- Frontend: package.json
- Esquema Prisma
- Motor de sorteio
- Testes de sorteio
- Express: arquivo server.ts
- Prisma: arquivo prisma.ts
- Componente SorteioDefinitivo
- Componente App
- Fluxo de participantes
- GitHub Actions: deploy.yml
- Licença MIT do repositório

Documentação de provedores e uso comercial

- GitHub: limites e proibições comerciais do GitHub Pages
- GitHub: termos adicionais para GitHub Pages
- Render: limites e suspensão da hospedagem gratuita
- Render: publicação de Web Services
- Prisma: documentação
- TiDB Cloud: documentação oficial
- React: documentação
- TypeScript: documentação
- Express: documentação

Conclusão. O projeto constitui uma base funcional e educativa de Full Stack, com um motor de sorteio superior a um embaralhamento simples por respeitar regras de atribuição e conflitos. A arquitetura demonstra separação adequada entre interface, API e dados, deploy automatizado e autenticação por perfis. Seu próximo salto não depende de adicionar mais tecnologias indiscriminadamente, mas de validar todo o fluxo do participante, formalizar o modelo de sigilo, estruturar proteção e operação e escolher infraestrutura contratualmente adequada ao uso comercial. O código-fonte integral deve sempre acompanhar este relatório como referência executável.